In-Vivo Fuzz Testing for Network Services

Wen-Yang Lai , Kun-Che Tsai, Che Chen, Yu-Sung Wu
Department of Computer Science
National Yang Ming Chiao Tung University
Hsinchu City, Taiwan
x3639026@gmail.com, q40603.cs05@nctu.edu.tw, che30122@gmail.com, ysw@nycu.edu.tw

***Abstract*—Fuzz testing is typically carried out by running the target program and the fuzzing engine offline in a lab environment. The environment setup may depend on specialized test harness code to activate the target program and inject the test data. Also, due to the vast program state space, domain knowledge-dependent optimization is often needed in the environment setup to achieve reasonably efficient fuzz testing. We propose In-Vivo Fuzzing to alleviate the burdens by performing online fuzz testing on live programs. In-Vivo Fuzzing hooks I/O library calls in a live program to collect test seeds. Upon request, the In-Vivo Runtime will create a fork of the target program and carry out fuzz testing on the forked process. The runtime states from the live program provide a vantage point to start the fuzzing process, and the test seeds collected from the live workload also facilitate the generation of effective test inputs. We applied In-Vivo Fuzzing to network service programs and implemented a prototype on top of the AFL fuzzer. Experiment results indicate that In-Vivo Fuzzing can reach vulnerabilities in real-world programs much more quickly than the baseline. We also demonstrate the potential application of In-Vivo Fuzzing in detecting unknown attacks, where live attack states are captured and amplified through fuzz testing.**

Keywords—Online fuzzing, live program, network services, passive fuzzing, production system, zero-day vulnerability, security isolation

I. INTRODUCTION

Software vulnerabilities remain one of the primary causes of security breaches, which leads to the deployment of defense mechanisms such as firewalls or access controls to block unsolicited access. At the same time, fuzz testing (or fuzzing)[1] has been widely employed to identify the presence of software vulnerabilities, where random (unexpected) inputs are applied to the software under test. A potential bug is identified if the software under test crashes or exhibits abnormal behaviors.

Given the random nature of fuzz testing and the vast program state space of software, it could actually take forever for a test to trigger a vulnerability of interest. In practice, fuzz testing is performed by experienced personnel in a lab environment. To begin with, the personnel needs to narrow down a region of the target software and prepare the test harness code to bridge the fuzzer with functions in the code region. The process is inevitably dependent on domain knowledge of the target software and experience in fuzz testing. Even so, the fuzz testing could take days to yield useful results, and further optimizations such as establishing realistic test environments [2-6], targeted test input generation [7-10], and addressing complex path conditions [11-13] are needed to achieve effective offline fuzz testing in lab environment.

We propose In-Vivo Fuzzing to realize online fuzz testing on live network service programs. With In-Vivo Fuzzing, a shadow copy (a child process fork or a container snapshot) of the network service process is created on-demand. The fuzz testing is carried out on the shadow copy, where the test data will be injected via the I/O function calls made by the shadow copy. In-Vivo Fuzzing automates the environment setup and does not depend on test harness code. As the shadow copy embodies the runtime states of the live program, In-Vivo Fuzzing can take a free ride with the existing workload to efficiently reach code locations of interest. In-Vivo Fuzzing could also take advantage of an ongoing attack attempt's (partial) attack payload to speed up the test input generation.

In-Vivo Fuzzing entails a unique set of challenges, including the collection of fuzzing seeds from live programs, the creation of the fuzzing instance (i.e., the shadow copy process), the automatic injection of test data, the security isolation of the fuzzing instance, and the selection of the fuzzing entry point. In the following, we will give an overview of In-Vivo Fuzzing in Sec. III. We will discuss the challenges and the design in Sec. IV–VI. The evaluation of the prototype is presented in Sec. VII. Discussions on related work and future work are given in Sec. VIII and Sec. IX respectively. The conclusion is given in Sec. X.

II. MOTIVATION

Traditional fuzz testing is performed in a detached environment separate from the production systems. The environment has to be set up appropriately to direct the fuzzer to the target region in the program under test [14] [15]. Likely, the random test input generation of a baseline fuzzer will get lost in the vast state space of a moderately-sized program. Techniques such as seed prioritization [8, 10], symbolic execution [16], program transformation [12], and program smoothing [11] have been explored to reduce the complexity in the search of effective input data. For network programs, the generation of test data also needs to take into consideration of the protocol states [4]. The technical barriers to applying fuzz testing in-house are actually higher than expected, and its practical use has been limited to experienced software testing engineers or software security professionals.

According to FireEye report [17], a new type of attack can remain undetected for up to 24 days (median dwell time). This can be seen as an indicator of how far the current software testing technologies fall behind the leading adversaries. The observation motivated the idea of deferring the fuzz testing process into the deployment and maintenance phases of the software lifecycle [18]. The approach comes with a bunch of advantages over traditional fuzz testing. First, the test can inherit the live program states from real-world workload to skip the initialization code and sanity checks [12, 16] on the code execution path leading to a potential vulnerability site. Second, the test can take the process structure and the live states of the target program as the basis of the test harness. Third, we can collect high-quality fuzz testing seeds from real-world input data in the production environment. Finally, when there is an ongoing attack on the system, the test can

2022 41st International Symposium on Reliable Distributed Systems (SRDS) | 978-1-6654-9753-4/22/$31.00 ©2022 IEEE | DOI: 10.1109/SRDS55811.2022.00014

2575-8462/22/$31.00 ©2022 IEEE
DOI 10.1109/SRDS55811.2022.00014

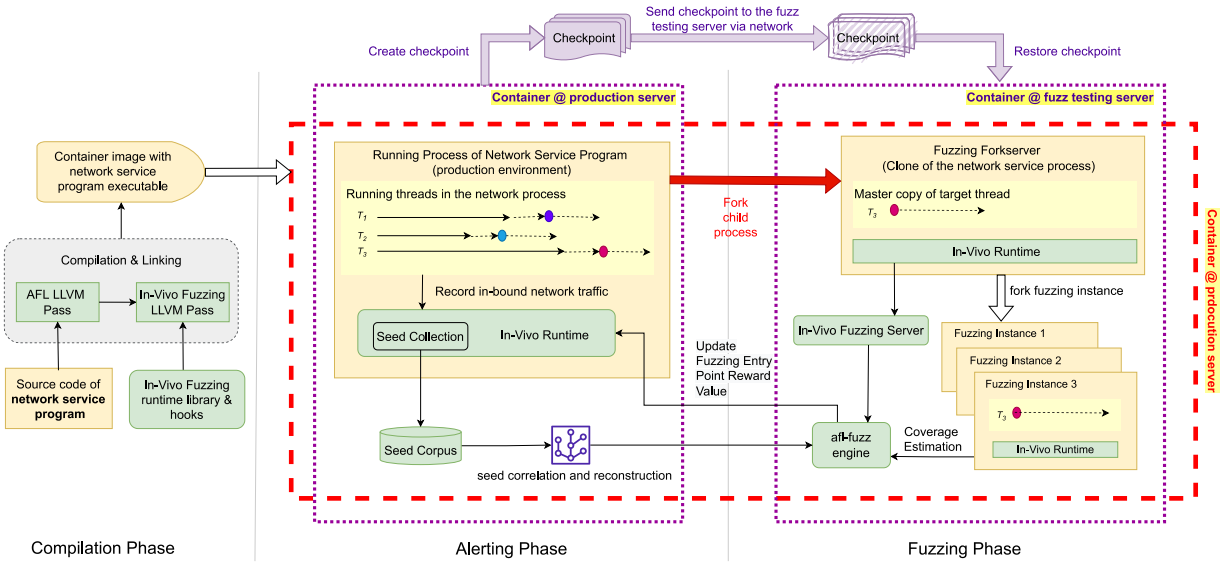

Fig. 1. In-Vivo Fuzzing Overview

potentially leverage the partial attack states to expedite the discovery of unknown vulnerabilities.

III. In-Vivo Fuzzing

We propose In-Vivo Fuzzing to realize online fuzz testing for live network service programs. The architecture of In-Vivo Fuzzing is given in Fig. 1. In-Vivo Fuzzing consists of three phases. In the *Compilation Phase*, we instrument the target program's executable with the AFL coverage estimation code, the In-Vivo Fuzzing Runtime library, and the In-Vivo hooks. Once the network program is deployed in the production environment, the system enters the *Alerting Phase*, in which the network service program is launched to handle the production workload. In the *Alerting Phase*, the instrumented In-Vivo Runtime Library will wait for fuzzing requests from the system administrator or security monitoring systems.

The *Fuzzing Phase* begins when a fuzzing request triggers the creation of the Fuzzing Forkserver, which is essentially a clone of the running process of the network program. The Fuzzing Forkserver will set up the security isolation boundary and send the fuzzing parameters (the pid of the Fuzzing Forkserver, the pid of the network service program, and the id of the shared memory used by the In-Vivo Runtimes) to the In-Vivo Fuzzing Server. The In-Vivo Fuzzing Server will start the afl-fuzz engine to perform fuzz testing on the child processes forked from the Fuzzing Forkserver. The child processes are referred to as the Fuzzing Instances. The fuzzing phase will terminate when the afl-fuzz engine reaches the maximum number of mutation iterations (MaxMutationIter).

Note that the alerting phase corresponds to running the network service program with In-Vivo Fuzzing instrumentation to serve the production workload that is supposed to be handled by the original network service program. After we initiate the fuzzing phase, the network service program in the alerting phase will continue to handle subsequent production workload. In other words, the processes in the fuzzing phase will be running concurrently with the network service program in the alerting phase.

The Fuzzing Forkserver, the fuzzing instances, the In-Vivo Fuzzing Server, and the afl-fuzz engine may be located on the same machine used for the alerting phase. This corresponds to the configuration marked by the rectangle shown in the red dashed line. This will be referred to as the *multi-process mode* in our follow-up discussions. We use seccomp [19] to implement the security isolation to prevent potential damages to the network service program (Sec. VI.D) in the *multi-process mode*.

If the production server is heavily loaded, it might be desirable to deploy the fuzzing phase in a dedicated fuzz testing server corresponding to the configuration marked by the rectangle in the purple dotted line. This is referred to as the *multi-container mode*. The prototype uses Podman[20] / CRIU[21] to checkpoint the container running the network service program in the alerting phase. The checkpoint will then be transferred to the dedicated fuzz testing server, where it will get restored to execute the subsequent steps of the fuzzing phase.

IV. Compilation Phase

The AFL fuzzing engine employed by the prototype supports QEMU-based dynamic instrumentation and LLVM-based instrumentation [22] for injecting coverage estimation code. We use the LLVM-based instrumentation to streamline additional In-Vivo Fuzzing-specific instrumentation. The instrumentation installs the In-Vivo Runtime in the target program, which collects the fuzz testing seed (Sec. V.A) and handles incoming fuzzing requests (Sec. VI.A) in the alerting phase. The In-Vivo Runtime is also involved in the estimation of the fuzzing entry point reward value (Sec. VI.B) in the fuzzing phase.

In addition, the instrumentation also installs the function entry point hooks for initiating the fuzzing phase (Sec. IV.B) and the hooks on selected I/O library calls. The hooks on I/O library calls will be used for seed collection in the alerting phase (Sec. V.A) and for maintaining the transparency of security isolation in the fuzzing phase (Sec. VI.D).

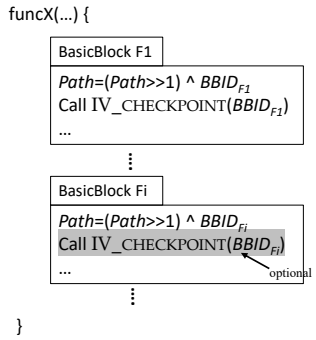

Fig. 2. Instrumentation of fuzzing entry points

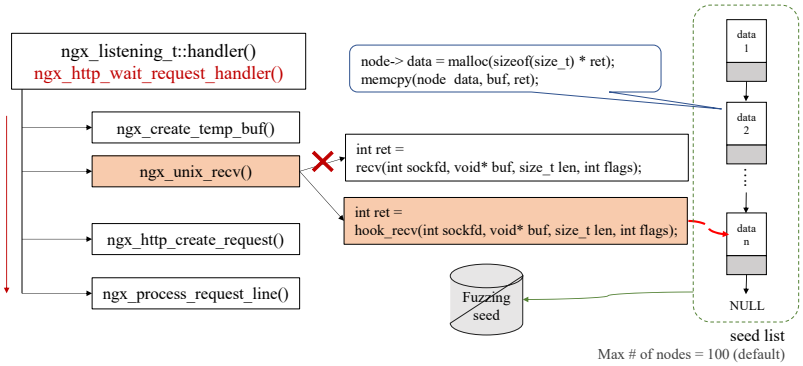

Fig. 3. Collection of fuzzing seeds

A. Wrapping External I/O Library Calls

We hook `read(2)`, `recv(2)`, `recvfrom(2)`, and `recvmsg(2)` to record the input data in the alerting phase for populating the fuzzing test seeds. In addition, the security isolation (Sec. VI.D) may lead to premature termination of a fuzzing instance when it makes an I/O call blocked by the security isolation. To address this issue, we also hook I/O library calls such as write(2), writev(2), sendto(2), and sendmsg(2) to emulate the semantics as if they are not blocked by the security isolation.

B. Instrumentation of Fuzzing Entry Point

The creation of the Fuzzing Forkserver involves a process fork from the live network service program. The initiating code for the process fork is packaged in the IV_CHECKPOINT() function of the In-Vivo Runtime library. We instrument a call to IV_CHECKPOINT() at the beginning of a function, as shown in Fig. 2. The call to IV_CHECKPOINT() is also referred to as a *fuzzing entry point* (Sec. V.B). After the In-Vivo Runtime receives a fuzzing request, the creation of the Fuzzing Forkserver will be initiated from the upcoming fuzzing entry point on the code execution path. One may instrument a call to IV_CHECKPOINT() in every basic block to minimize the latency of the process fork. However, this will increase the target program's performance overhead in the alerting phase. On the other hand, one may choose to install IV_CHECKPOINT() only on a subset of the functions in the network service program (e.g., skip those known to be stable) to reduce the instrumentation overhead.

The In-Vivo Runtime maintains a path-dependent reward value map on each fuzzing entry point (Sec. VI.B) to prioritize the selection of appropriate fuzzing entry points. During the instrumentation, we also instrument the path ID code (a shift and xor operation with the basic block ID) at the beginning of each basic block, as shown in Fig. 2.

V. Alerting Phase

The network service program executable with In-Vivo Fuzzing instrumentation will be deployed to the production environment and begin to serve the incoming workload as usual. This corresponds to the alerting phase, during which the In-Vivo Runtime in the network service program waits for incoming fuzzing requests to initiate the fuzzing phase. The In-Vivo Runtime also records the input data (e.g., network packet payload) from the I/O calls performed by the network service program in the alerting phase to populate the test seed corpus.

A. Recording Inbound Network Traffic

The In-Vivo Runtime hooks selected networking I/O calls in the network service program to record the received packets as fuzzing seeds. We use fstat(2) to distinguish network socket descriptors. As shown in Fig. 3, the packet data received from each call to read(2), recv(2), or recvmsg(2) are stored as a node in the seed list. One can use the compile-time constant *MAX_IV_BUFFER* (default: 100 nodes) to control the maximum number of nodes in the seed list (the oldest one will be purged when the list is full).

B. Fuzzing Request and Creation of Fuzzing Fork Server

As part of the instrumentation, we enable the GNU C constructor function attribute [23] on the initialization function of the In-Vivo Fuzzing Runtime Library. The initialization function will be invoked at program load time to spawn a thread for handling incoming fuzzing requests and fuzzing entry point reward value updates (Sec. VI.B).

TABLE I In-Vivo Fuzzing Communication Packet (Fuzzing Request)

Field	Data Type	Description
request_type	int	Set to value 1
pid	Int	The ID of the target process for In-Vivo Fuzzing
tid	int: 16	The ID of the target thread for In-Vivo Fuzzing
bbid	int: 16	The ID of the basic block to start the Fuzzing Forkserver (set to 0 to choose any basic block)

An In-Vivo Fuzzing Request can be issued manually by the system administrator or automatically by an anomaly detection system. In-Vivo Fuzzing Requests follow the In-Vivo Fuzzing Communication Packet format (TABLE I). We can restrict the selection of the fuzzing entry point by setting the bbid field, so the fuzzing phase will only start at the fuzzing entry point on the basic block (i.e., the Fuzzing Forkserver will be created by the call to IV_CHECKPOINT() at the basic block). If the target program is multi-threaded, we can also use the tid field to designate the target thread for the creation of the Fuzzing Forkserver.

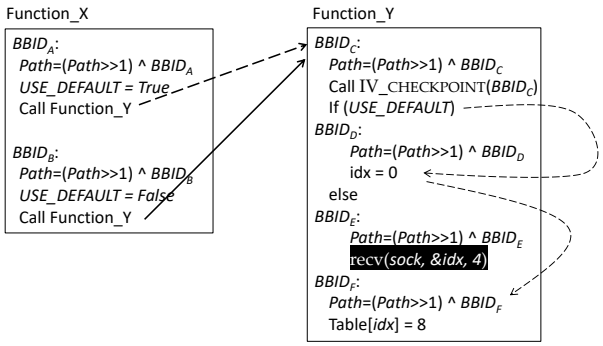

Fig. 4. Path-dependent fuzzing entry point filter

C. Path-dependent Fuzzing Entry Point Filter

Different code execution paths may lead to the same basic block. The effectiveness of fuzz testing from an entry point may depend on the program states accumulated on the preceding execution path. For instance, when we initiate In-Vivo Fuzzing from the `Function_Y` in Fig. 4, a potential buffer overflow in basic block E can be triggered by external input provided that the value of *`USE_DEFAULT`* is false, which requires the code execution path in `Function_X` to have gone through basic block B.

During the instrumentation, we embed the `Path` tracking code at the beginning of each basic block to model the recent history of the code execution path. In IV_CHECKPOINT(), we use `Path` as the key to retrieve the reward value of the path in the BSA_entry_value_map map, where a large path reward value indicates a preferable code execution path. In the alerting phase, we can set the threshold value MinReward to filter out undesirable fuzzing entry points (Line 7 in Fig. 5).

```
function IV_CHECKPOINT(bbid)
▷ fq is the fuzzing request received by the In-Vivo Runtime
▷ Path is a per-thread global variable that models the (recent)
  history of executed basic blocks
▷ BSA_entry_value_map is a per-thread global variable tracking
  the path reward values (default value in each entry: ∞)
▷ MinReward is the minimum reward value needed for a
  fuzzing entry point to be selected
▷ bbid is the id of the basic block where the IV_CHECKPOINT() is
  instrumented
▷ Phase indicates whether the program is running in the
  alerting phase or the fuzzing phase
1:   If Phase = ALERTING_PHASE
2:       If fq=NIL return
3:       If fq.tid != 0 and fq.tid != CURRENT_THREAD_ID()
4:           return
5:       If fq.bbid != 0 and fq.bbid != bbid
6:           return
7:       If BSA_entry_value_map[Path] < MinReward
8:           return
9:       CREATE_FUZZING_FORKSERVER()
10:  Else // FUZZING_PHASE
11:      If TERMINATION[Path] = TRUE
12:          EXIT()  // terminate the process
```

Fig. 5. Pseudocode of IV_CHECKPOINT

VI. FUZZING PHASE

Upon receiving a fuzzing request, the In-Vivo Runtime will ready the fuzzing seeds and set up the fuzzing request global variable fq. When the network service program's execution reaches a basic block with a matching IV_CHECKPOINT() call, the Fuzzing Forkserver will be created by forking a child process from the network service program (Line 9 in Fig. 5). The In-Vivo Runtime in the Fuzzing Forkserver process will proceed to launch the In-Vivo Fuzzing Server to orchestrate the operations in the fuzzing phase. Once started, the In-Vivo Fuzzing Server will create private copies of the shared memory segments inherited from the network program in the Fuzzing Forkserver. The In-Vivo Fuzzing Server will also set up seccomp rules to isolate the Fuzzing Forkserver. At last, the In-Vivo Fuzzing Server will invoke the AFL-fuzz engine to create the fuzzing instance and carry out the fuzz testing.

A. Preparation of Fuzzing Seeds

In-Vivo Fuzzing records network traffic from the live network service program to populate the fuzz testing seeds. Upon receiving a fuzzing request, the data on the seed list (Fig. 3) will be dumped into the AFL input seed directory. As the seeds are derived from the production workload, the number of mutations on the test input needed to reach a vulnerability can be significantly reduced. For the injection of test input, we replace the socket file descriptors in the fuzzing instance with domain sockets and direct the output from the AFL mutation engine to the domain sockets.

B. Reward Value of Fuzzing Entry Point & Coverage Estimation

In Sec. V.C, we introduced the path-dependent fuzzing entry point filter BSA_entry_value_map[Path] to regulate the activation of the fuzzing entry point in IV_CHECKPOINT(). One may assign a zero value to BSA_entry_value_map[Path] (or any value below the threshold MinReward) to avoid initiating In-Vivo Fuzzing from the fuzzing entry point reached via Path. The BSA_entry_value_map[Path] records the number of subsequent unique paths found by AFL in the previous round of In-Vivo Fuzzing from the fuzzing entry point on Path. At the end of each round of fuzzing, AFL will calculate the number of unique paths found and report it back to the In-Vivo Runtime in the alerting phase (note that the network program continues its execution in the alerting phase after the creation of the Fuzzing Forkserver). The format of the reward report packet is shown in TABLE II, where the reward value field corresponds to the number of unique paths found.

TABLE II. In-Vivo Fuzzing Communication Packet (Coverage Reports)

Field	Data Type	Description
Packet Type	int: 16	Set to value 2
Path ID	int: 16	The ID of the path to be updated.
Reward Value	int: 16	The new reward value

C. Termination of Fuzzing Instance

One may think of the execution of a network service program as a loop of I/O operations and processings. The current In-Vivo Fuzzing prototype only injects test data on established network connections. As a result, the execution of

the fuzzing instance will be blocked on a call to socket listen(2). To improve the throughput of the fuzzing phase, we maintain the termination record table TERMINATION[PATH] in the Fuzzing Forkserver. Each table entry is a boolean value corresponding to the given PATH. A TRUE value indicates the execution path had been blocked in previous runs and should be avoided in future runs. We currently use timeouts to detect blocked executions and set the corresponding entries in TERMINATION[PATH] to TRUE. When a fuzzing instance terminates abnormally, the Fuzzing Forkserver will collect the process state from waitpid(2) and report the value to the afl-fuzz engine.

TABLE III. Examples of system calls blocked via seccomp

sendmsg	epoll_ctl	write
sendfile	setsockopt	sendtimedop
epoll_create	semget	writev
epoll_wait	semop	

D. Security Isolation

For security, the fuzzing instances are sandboxed from the rest of the system. In the *multi-process mode*, the fuzzing instances co-locate with the network service process in the alerting phase. For security isolation, we close the descriptors of shared system resources on the fuzzing instances by default and replace the descriptors of essential resources with descriptors pointing to newly allocated private copies. Shared memory segments are replaced with private copies as well. Finally, we use seccomp to block unwanted system calls (TABLE III) from the fuzzing instances. In the *multi-container mode*, the fuzzing instances are encapsulated in a container running on a dedicated fuzz testing server. We rely on the container and the physical isolation of the dedicated server for the security isolation in the *multi-container mode*.

It is noteworthy that the security isolation has to maintain a transparent runtime environment for the fuzzing instances to achieve decent fuzzing coverage. For instance, a fuzzing instance may attempt to perform a write to a shared resource that is disallowed by the sandbox mechanism. If we fail the write operation, the execution of the fuzzing instance could terminate prematurely, preventing the test from reaching deeper code. To maintain transparency, we hook the I/O library calls of the fuzzing instances (Sec. IV.A) and emulate the semantics of the calls (e.g., faking a successful write to the shared resource) to avoid premature termination.

VII. EVALUATION

We evaluate the In-Vivo Fuzzing prototype on different types of network services along with real-world CVEs as summarized in TABLE IV. We categorize the CVEs by whether they are preceded by a series of stateful protocol handshakes (CCF) and whether they depend on input data in a complex message format (CI). Both attributes are challenging for the baseline AFL. The experiment platform is equipped with Intel(R) Core(TM) i7-4790 CPU with 32G RAM. In Sec. VII.A, we measured the runtime overhead on the network service programs in the alerting phase caused by the instrumentation. In Sec. VII.B, we compared the usage of In-Vivo Fuzzing vs. AFLNet. In Sec. VII.C, we measured the time needed to trigger the vulnerabilities with different configurations of In-Vivo Fuzzing. In Sec. VII.D, we took a close look at the cases of Nginx, MySQL, and Pure-ftpd to quantify the impact of the choice of fuzzing entry points.

TABLE IV. List of network services and CVEs

Program	CVE	CCF	CI
MySQL-5.6.25	CVE-2017-3599	v	
Httpd-2.4.25	CVE-2017-7659		v
Nginx-1.4.0	CVE-2013-2028		v
Redis-5.0-rc1	CVE-2018-12453	v	
Exim- 4.92.1	CVE-2019-16928		v
Live555-0.92	CVE-2018-4013	v	
Pure-ftpd	CVE-2020-9365		v
ntp-4.2.8p8	CVE-2016-7434		v
tinydtls-0.8.2	CVE-2017-7243		v
Memcached-1.4.3	CVE-2017-9951		v

(CCF = COMPLEX CONTROL FLOW, CI=COMPLEX INPUT)

A. Runtime Overhead of Alerting Phase

We measured the execution time of MySQL, Nginx, Httpd, Redis, and Memcached under three different instrumentation configurations. The first is the standard AFL instrumentation (instrumented by the afl-clang-fast command). This is denoted as 'AFL Clang' in Fig. 6. The second configuration combines the standard AFL instrumentation with the instrumentation of IV_CHECKPOINT(). This is denoted as 'AFL Clang + IV_CHECKPOINT.' The third configuration adds the collection of fuzzing seeds on top of the second configuration. This is denoted as 'AFL Clang + IV_CHECKPOINT + Seed Collection.' For the benchmark workload, we use wrk [24] and ab for Nginx and Httpd, respectively. Both were set to use 4 threads, 1000 connections, and an execution time of 40 seconds. For MySQL, we use sysbench [25] with 4 threads and 10,000 requests. For Redis, we use the built-in benchmark program redis-benchmark with 50 parallel connections and 100,000 requests. For Memcached, we use memslap with 100,000 requests.

As shown in Fig. 6, we can see that the In-Vivo instrumentation incurs about 20% performance overhead for Nginx, Redis, Httpd, and Memcached. The instrumentation of IV_CHECKPOINT() adds a tiny amount of overhead on top of the coverage estimation code from standard AFL instrumentation. The overhead from the collection of fuzzing seeds is mild because the seed list is kept in the memory and tracks only the most recent 100 records. The performance overhead of MySQL rises to 130% because the processing of each SQL query involves a large number of function calls, which amplifies the overhead of the AFL per-basic block coverage estimation code instrumentation.

We also experimented with running 12 fuzzing instances in multi-process mode with the network service programs in the alerting phase to observe the performance impact of running co-located fuzzing instances. This is denoted as 'AFL Clang + IV_CHECKPOINT + Seed Collection + 12 Fuzzing Instances' in Fig. 6. Interestingly, the co-located fuzzing instances did not cause much extra overhead. This is because the fuzz testing is an I/O intensive operation (due to the disk writes involved in the seed mutation process), while the network service programs can serve most of the workload

requests with little disk I/O. On the other hand, the I/O bottleneck indicates the need to use the multi-container mode to offload the fuzzing instances to machines equipped with fast disks.

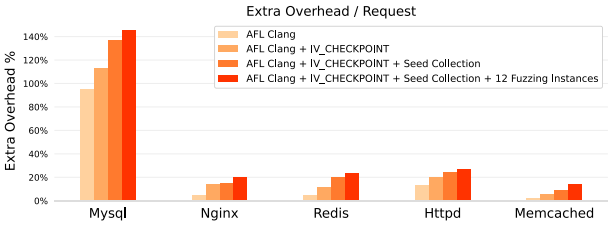

Fig. 6. Extra performance overhead on network service programs in alerting phase

B. *Usage Comparison of In-Vivo Fuzzing vs. AFLNet*

Here we take Pure-ftpd with CVE-2020-9365 as an example to demonstrate the differences in the usage between In-Vivo Fuzzing and offline fuzzing. For offline fuzzing, we use AFLNet [4], which is a specialized version of AFL for testing network protocols. The FTP protocol used by Pure-ftpd is among the network protocols supported by AFLNet. The CVE is due to an out-of-bounds access in the pure_memcmp function, which will be invoked in the verification of FTP login password when using MySQL database to manage the user account information of Pure-ftpd. The CVE is triggered by first sending the username via the USER command, followed by sending a password with excessive length with the PASS command to trigger the out-of-bounds access. It is noteworthy that the authentication process is stateful. If the username does not match the record of an existing account, the Pure-ftpd will ignore the supplied password and reject the connection.

Running AFLNet on the Pure-ftpd without any manual setup could not trigger the vulnerability (We terminated the experiment after running the test for 6 hours). Knowing that AFLNet might have got stuck in the authentication process, we manually added the username string of an existing account in the AFLNet dictionary and carried out the test again. AFLNet was able to trigger the vulnerability after 3 hours with the help of the username in the dictionary.

In comparison, In-Vivo Fuzzing automatically records the username of existing accounts as seeds from the live FTP sessions. By selecting the fuzzing entry point located right in front of the socket library calls to receive the username and password, the In-Vivo Fuzzing triggered the vulnerability in just 1 second. It is noteworthy that the selection of the fuzzing entry point was also effortless. If we use the trial-and-error method to progress the selection of the fuzzing entry point, it would only take six trials (Fig. 10) to reach the optimal choice. It is also noteworthy that In-Vivo Fuzzing is application-agnostic and protocol-agnostic. There was no special treatment for the FTP protocol or the Pure-ftpd server in using In-Vivo Fuzzing. On the other hand, AFLNet would require even more setup efforts besides the preparation of the dictionary entries when testing an unsupported protocol.

C. *Effectiveness of In-Vivo Vulnerability Discovery*

Here we applied In-Vivo Fuzzing to 10 representative network service programs (TABLE IV) and measured the time taken by the fuzzer to trigger the vulnerabilities. The network service programs cover the essential services on today's network, including two web servers, one relational database server, two NoSQL datastores, one mail server, one streaming server, one file server, one transport encryption library (with the built-in test server), and one clock synchronization server.

As summarized in TABLE V, we carried out In-Vivo Fuzzing on each program with three different configurations. With configuration [C1. Fuzzing Entry Point], we initiated the fuzzing phase from the chosen entry point with an empty seed (no seed collection at the alerting phase). With configuration [C2. Fuzzing Entry Point + Normal Seed Only], we applied normal workload to the network service and collected the fuzzing seeds in the alerting phase. Right after that, we initiated the fuzzing phase from the chosen fuzzing entry point with the seeds. With configuration [C3. Fuzzing Entrypoint + Quasi-Attack Seed], we mixed the quasi-attack workload with the normal workload in the alerting phase and then collected the fuzzing seeds. After that, we initiated the fuzzing phase from the chosen fuzzing entry point with the seeds. The quasi-attack was generated with weakened versions of exploit code that cannot trigger the vulnerabilities, which simulates a close but unsuccessful attack attempt.

In the following, we take *MySQL*, *Exim*, and *Redis* as examples to elaborate on the collection of fuzzing seeds and the considerations behind the selection of fuzzing entry points. The experiment results are summarized in TABLE V.

1) MySQL: For the MySQL 5.6.X branch up to 5.6.35 and the 5.7.X branch up to 5.7.17, there exist an integer overflow vulnerability in the function parse_client_handshake_packet (CVE-2017-3599), which allows remote attackers to launch a denial of service attack on the server with a malformed authentication packet. MySQL uses the my_net_read function to receive network packets. After a series of calls (my_net_read→net_read_packet→net_read_packet_header→net_read_packet_header→net_read_raw_loop→vio_read→inline_mysql_socket_recv→recv), it will reach the recv(2) call, through which the attack payload will be injected to the MySQL server program.

For the selection of fuzzing entry point, choosing any function prior to the recv(2) call would suffice. However, a limitation is that the current prototype does not emulate the establishment of network connections, and the execution of the fuzzing instance would be blocked on a call to socket listen(2). Preferably, we should choose a fuzzing entry point between listen(2) and recv(2). In this experiment, we chose my_net_read as the fuzzing entry point.

In the alerting phase, we performed a couple of normal MySQL logins for the In-Vivo Runtime to collect the login packets as the normal seed. To collect the quasi-attack seed, we mixed the normal workload with the login packets generated by a weakened version of the CVE-2017-3599

exploit code. The login packets from the weakened exploit code cannot trigger the vulnerability.

2) Exim: Exim is a message transfer agent on Unix-like systems. Exim 4.92 through 4.92.2 exhibit the CVE-2019-16928 vulnerability. The vulnerability is due to a heap buffer overflow at string_vformat in string.c. The attacker can send a long EHLO command to overflow the buffer and trigger remote code execution. Exim spawns a child process to handle each incoming SMTP connection. The primary handler function is smtp_setup_msg(), so we chose it as the fuzzing entry point. For the seed collection, we used a script to generate a bunch of SMTP connections as the normal workload. The quasi-attack workload includes some short EHLO strings in the SMTP connections (note that only a long EHLO string would trigger the vulnerability).

3) Redis: Redis(Remote Dictionary Server) is an in-memory NoSQL datastore. There is a type confusion in the xgroup command function in t_stream.c before Redis 5.0. Remote attackers can launch a denial-of-service attack via an XGROUP command with an incorrect key type. In the alerting phase, we applied a normal workload consisting of SET, GET, DEL, and XGROUP commands for seed collection. We entered the fuzzing phase from the fuzzing entry point readQueryFromClient. As indicated in TABLE V, the vulnerability can be triggered in 2 minutes with the normal seed, so we did not carry out a separate experiment with the quasi-attack seed.

The experiment results are summarized in TABLE V. We can see that In-Vivo Fuzzing with a proper selection of fuzzing entry point and fuzzing seeds (C2 and C3) outperforms In-Vivo Fuzzing without using any seed (C1). Note that the prototype employs AFL as the fuzzing engine. The time needed to reach vulnerabilities with plain AFL will be longer than C1.

In the cases of Nginx and Redis, In-Vivo Fuzzing was able to trigger the vulnerabilities within minutes with normal seeds (C2). In the other cases, the quasi-attack seeds (C3) were crucial to reducing the fuzzing time down to the minutes. We see that fuzz testing on live programs can reach vulnerabilities with less effort than traditional offline fuzz testing. The experiments with quasi-attack seeds show the potential of repurposing fuzz testing for detecting zero-day attacks, where the In-Vivo Fuzzing alerting phase captures the unknown attack workload as test seeds and ascertain its risk through the fuzzing phase.

```
▷ Assume MinReward > 0
▷ Instrumentation of fuzzing entry point only at the first basic
  block in each function
▷ NumRun is the number of experiments. Each experiment
  uses a distinct fuzzing entry point
1:  ∀Path, BSA_entry_value_map[Path] ← MinReward
2:  while NumRun > 0
3:      Create a new fuzzing request fq
4:      fq.tid ← {ID of the target thread}
5:      fq.bbid ← 0
6:      Send fq to the target program in alerting phase
7:      wait till the fuzzing phase starts
8:      // Assume the fuzzing phase starts from the fuzzing
          entry point (IV_CHECKPOINT) on the execution
          path PathTaken
9:      BSA_entry_value_map[PathTaken] ← 0
10      NumRun ← NumRun - 1
11      wait till the fuzzing phase is completed
```

Fig. 7. Repeating In-Vivo Fuzzing from a progressively selected fuzzing entry point

D. Effect on the Choice of Fuzzing Entry Point

To evaluate the effect on the choice of fuzzing entry point, we carry out In-Vivo Fuzzing from different fuzzing entry points for *Nginx*, *MySQL*, and *Pure-ftpd*. In the alerting phase, we collect both the normal workload seeds and the quasi-attack seeds as in Sec. VII.C. To exercise different choices of fuzzing entry points, we use the path-dependent entry point filter BSA_entry_value_map[Path] to progress the selection of fuzzing entry points in each experiment. The process flow is given in Fig. 7. At Line 1, we initialize the reward value on every path with MinReward, which basically means any fuzzing entry point can be used to launch the fuzzing phase. Each loop iteration at Line 2 corresponds to one experiment, and each experiment begins with a new fuzzing request (Line 3). At the end of an experiment, we will set BSA_entry_value_map[Path] to 0 (Line 9), so in the next experiment, we will try a new path and use the fuzzing entry point ending on the new path. This is how we progress the selection of fuzzing entry points throughout the experiments. For the fuzzing phase, we allow AFL to perform at most 500,000 mutations. At the end of the fuzzing phase (Line 11), we measure the number of unique paths and the number of unique crashes found by AFL.

TABLE V. Fuzzing time for reaching vulnerability site ('X' indicates the fuzzer were not able to reach vulnerability in the experiment duration)

Program	CVE	Fuzzing Entry Point (function)	C1. Fuzzing Entry Point	C2. Fuzzing Entry point + Normal Seed Only	C3. Fuzzing Entry Point + Quasi-Attack Seed
MySQL-5.6.25	CVE-2017-3599	my_net_read	X	15 hr 26 min 29s	8 min 43s
Httpd-2.4.25	CVE-2017-7659	socket_bucket_read	X	1 hr 36 min 11s	2 min 21s
Nginx-1.4.0	CVE-2013-2028	ngx_unix_recv	25 min 33s	8 min 17s	3 min 02s
Redis-5.0-rc1	CVE-2018-12453	readQueryFromClient	X	2 min 09s	2 min 09s (normal seed)
Exim- 4.92.1	CVE-2019-16928	smtp_setup_msg	X	X	1 day 3 hr 56s
Live555-0.92	CVE-2018-4013	handleRequestBytes	X	3 hr 58 min 27s	21 min 16s
Pure-ftpd	CVE-2020-9365	sfgets	X	X	650ms
ntp-4.2.8p8	CVE-2016-7434	read_network_packet	X	13 hr 46 min 27 s	5 hr 57 min 43 s
tinydtls-0.8.2	CVE-2017-7243	dtls_handle_read	X	X	15 min 19 s
Memcached-1.4.3	CVE-2017-9951	tcp_read	X	X	4 hr 16 min 57s

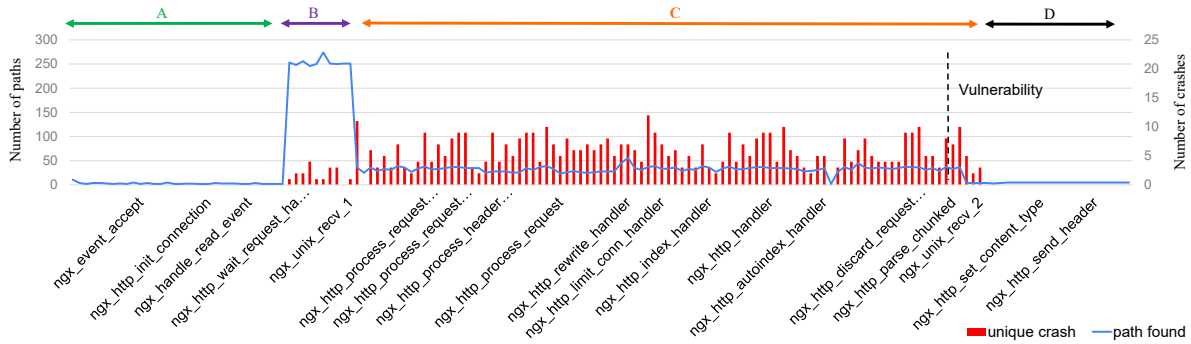

Fig. 8. Impact of vulnerability proximity on fuzzing effectivenes (Nginx / CVE-2013-2028)

In Fig. 8, Fig. 9, and Fig. 10, we plot the numbers for the fuzzing entry point selected in each experiment for Nginx, MySQL, and Pure-ftpd, respectively. The function name of the chosen fuzzing entry point and the numbers are plotted in the order of the experiments. For instance, in Fig. 8, we used the ngx_event_accept as the fuzzing entry point in an earlier experiment, followed by ngx_http_init_connection as the fuzzing entry point in a later experiment. To avoid cluttering labels in the plots, we only show a subset of the fuzzing entry point names on the x-axis. To further improve the readability of the figures, we only show the function where IV_CHECKPOINT() was activated. For instance, assume that the fuzzing phase in an experiment begins with the creation of the Fuzzing Forkserver at the IV_CHECKPOINT() in FuncY(), where the call stack (execution path) is FuncW() →FuncX()→**FuncY()**→**FuncZ()**. We will only put the label 'FuncY' on the x-axis. In the case of multiple execution paths leading to the same function that activates IV_CHECKPOINT(), we add tail numbers to distinguish them (e.g., FuncY_1, FuncY_2, …). The vertical dashed line on the plot denotes the function containing the respective vulnerability.

1) Nginx: We set up Nginx-1.4.0 and direct In-Vivo Fuzzing towards the CVE-2013-2028 vulnerability. A specially-crafted chunked Transfer-Encoding request can trigger an integer signedness error and lead to a stack-based buffer overflow in the Nginx server. In the alerting phase, we apply benign HTTP workload (some of the connections involve normal chunked Transfer-Encoding) to the Nginx server for the In-Vivo Runtime to collect the fuzzing seeds.

The number of unique paths and the number of crashes from initiating the fuzzing phase at each fuzzing entry point are given in Fig. 8. In section A, the number of unique paths and the number of crashes are low because the HTTP connections had not been properly established, and no test data input could be injected via the network API calls to the Nginx server by AFL. In section B, starting from ngx_http_wait_request_handler to ngx_unix_recv_1, the Nginx server receives the HTTP request header packets containing the Transfer-encoding field, and the number of unique paths reaches the peak accordingly. In section C, between ngx_unix_recv_1 and ngx_unix_recv_2 (via ngx_http_read_discarded_request_body), the number of unique paths goes down, and the number of unique crashes found rises up. This is because the input to ngx_unix_recv_1 constrained the control flow of the Nginx server to the processing of the HTTP header information (so the number of unique paths drops), and the afl-fuzz engine could focus on exploring the test input to ngx_unix_recv_2 for triggering the vulnerability (so the number of crashes goes up). In section D, the number of unique paths decreases sharply, and the number of crashes drops to zero after ngx_unix_recv_2 because no network I/O calls are present on the code path following the fuzzing entry points in this region.

2) MySQL: The MySQL / CVE-2017-3599 experiment results are presented in Fig. 9. In section A, a network connection had not been established. The fuzzing entry points such as waiting_thd_list(), pop_front(), add_global_thread(), and thd_prepare_connection() are used for resource allocation and initialization prior to the establishment of network connection with the MySQL client. In-Vivo Fuzzing from these entry points was ineffective because the code execution was blocked by the wait for incoming network connections.

In section B, after establishing a network connection, the MySQL server proceeds with the authentication steps with acl_check_host, acl_authenticate, and do_auth_once. The numbers of unique paths explored by AFL are consistently high in this section, as the entry points can lead to the injection of test data. The numbers of crashes are particularly intense near the end of section B because the fuzzing entry points near the end are closer to the vulnerable code location, which had a higher chance of triggering a crash as we limited the total number of mutations in AFL to MaxMutationIter (500,000) per experiment. In section C, though AFL could still explore new paths through another my_net_read invoked from the fuzzing entry points in the section, none of these tests would trigger the integer overflow vulnerability because the selected fuzzing entry points could not reach the vulnerable code site in parse_client_handshake_packet(). In section D, there were no I/O library calls available for injecting test data, so both the number of paths and the number of crashes dropped to zero.

3) Pure-ftpd: The experiment result for Pure-ftpd is given in Fig. 10. The number of unique paths and the number of unique crashes reach the peaks at the fuzzing entry point

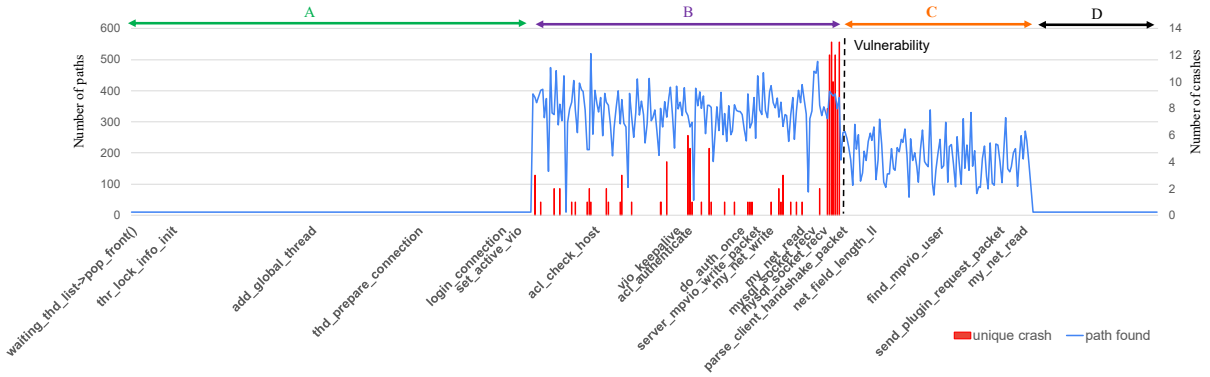

Fig. 9. Impact of vulnerability proximity on fuzzing effectiveness (MySQL / CVE-2017-3599)

sfgets, which is the only location in the source code of *Pure-ftpd* for receiving data on the command channel and is exploited by CVE-2020-9365 for injecting malicious payload. The fuzzing entry points in section C cannot reach sfgets, so the fuzzer could not explore new paths or crashes.

After the fuzzing entry point pure_memzero (at the end of section C), Pure-ftpd continues to fetch the next command from the client as we apply the workload to Pure-ftpd in the alerting phase. This leads to the fuzzing entry points in section B' and section A', which are similar to section B and section A, respectively. The same vulnerability can be triggered again from the entry points in B' and A'. We omit showing the vulnerability dash line (to the right side of section A') again on the plot for readability. The fuzzing entry points in both section A and section B can reach sfgets, and the number of unique paths and the number of crashes are noticeably higher than in the other regions. The numbers of unique paths and crashes in section A are higher than in section B because the fuzzing entry points in section B resulted in higher edge coverage (due to the longer path leading to sfgets), causing AFL to assign a higher score to the initial seed. Consequently, it was more difficult for the new seeds to become favorite seeds, and AFL tends to prefer deterministic mutation over havoc mutation in this case. The deterministic mutation made it more difficult to uncover new paths and crashes. And, as we limit the total number of mutations iteration to 500,000, AFL could not adapt its mutation strategy in time to yield the same numbers of unique paths and crashes.

Overall, we can see that the choice of fuzzing entry points clearly impacts fuzzing efficiency, including the number of discovered paths and the number of unique crashes. In general, it is advantageous to initiate fuzz testing from locations before I/O calls. It would be even better to focus on locations where the input variables are tightly coupled with subsequent control flows. As a future work on the automatic selection of fuzzing entry points, we may prioritize the selection of fuzzing entry points with imminent data inputs (via the help of static analysis) and high runtime error rates (e.g., segmentation fault events).

VIII. Related Work

The random test input generation in baseline fuzzing is rather inefficient in dealing with complex branch conditions in a program. Driller [16] employs selective symbolic execution to generate inputs to satisfy complex branch conditions. Vuzzer [7] leverages control- and data-flow features in the target program to generate inputs that can reach deeper code paths. T-Fuzz [12] improves the scalability of the fuzzing process with binary rewriting that bypasses complex condition checks and subsequent constraint solving to reconstruct the complete inputs. NEFUZZ [11] creates a smooth surrogate function approximating the target program's discrete branching behavior and uses gradient-guided input generation schemes to tackle complex branch conditions. Most of the existing work can be integrated with In-Vivo Fuzzing. The treatments of path conditions [11, 16] and the improvement of coverage estimation [8] would benefit the selection of fuzzing entry points. The handling of application-

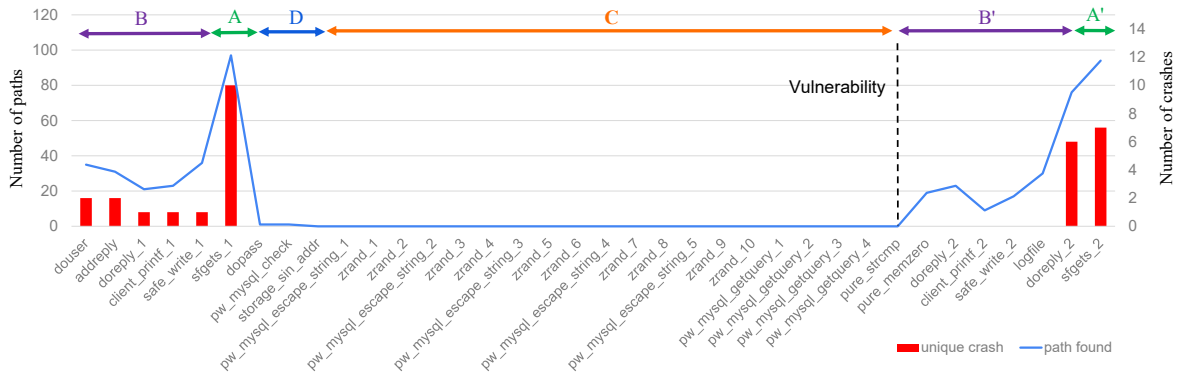

Fig. 10. Impact of vulnerability proximity on fuzzing effectivness (Pure-ftpd / CVE-2020-9365)

specific input data [7, 26] can be integrated into the seed collection in the alerting phase. The symbolic execution-based post-processing step in T-Fuzz [12] can be extended to generate the complete input sequence (in particular, those inputs that happened before the chosen fuzzing entry point) needed to reproduce the bug from a fresh run of the target program.

The input interface of a real-world program can take different forms. For instance, kernel programs or library programs do not interact directly with standard input or files in general. A test harness is required to inject the test data appropriately. DIFUZE [26] employs static analysis to compose correctly-structured input in the user space and inject the test data via ioctl to kernel drivers. Razzer [6] implements deterministic thread interleaving with a hypervisor and uses multi-thread fuzz testing to identify kernel data race bugs. FuzzGen [14] automates the generation of fuzzer stubs for testing library functions via static analysis of existing application code that calls the library functions. For library developers, it is possible to embed the fuzzing interface into a library with libFuzzer [15]. Overall, the injection of test data is non-trivial for real-world programs. The use of compile-time hooks for test data injection in In-Vivo Fuzzing provides a generic way to cope with the diverse program input interfaces. The challenge of generating a correctly-structured input sequence is partly addressed by the seed collection from the live workload in the alerting phase. On the other hand, network services can also suffer from race bugs. In-Vivo fuzzing can potentially be extended to generate concurrent test data, as in Razzer [6].

Remotely exploitable vulnerabilities in network services have led to an interest in applying fuzz testing on network services. Prenny [27] provides a library that redirects standard input to network input, making it easier to carry out fuzz testing on network services locally. AFLNet [4] is a grey-box protocol fuzzer based-on AFL, which can generate stateful test data for RTSP, FTP, DTLS12, DNS, DICOM, SMTP, SSH, and TLS protocols. Based on the estimation of protocol states with the number of packet flights seen in each protocol execution, Yurong [28] proposes a progressive stateful protocol fuzzer to capture the state changes in networking protocols and heuristically explore code spaces that are related to multiple protocol states. By leveraging the live runtime states of network service programs, In-Vivo Fuzzing deals with the network protocol states more generically. On the other hand, knowledge of the protocol specifications and protocol state-aware test input generation are both compatible with the In-Vivo Fuzzing architecture. In particular, the collection of fuzz testing seeds can be linked with the network protocol states to reduce the overhead of redundant seeds.

Data dependency and control dependency in large programs can make it quite difficult to rely on the test harness code to initialize the states of the target program properly for fuzz testing. One notable example is the OS kernel. This leads to passive fuzzing techniques [29-31], where the fuzz testing engine intercepts and mutates the communications (e.g., system calls) between the application workload and the OS kernel. X-afl [32] further extends passive fuzzing by integrating an active fuzzer engine to leverage code coverage information and improve test reproducibility. The above work all target the OS kernel, while In-Vivo Fuzzing targets network services. The injection of test data in passive fuzzing is performed through the system call or ioctl interfaces, while In-Vivo Fuzzing intervenes at the library call interface. It is noteworthy that the passive fuzzing was carried out with synthetic application workloads in a lab environment. In contrast, In-Vivo Fuzzing operates directly on the production software leveraging the real-world workload.

IX. Future Work

The activation of the fuzzing phase can be automatically triggered by an anomaly detection system. For instance, In-Vivi Fuzzing can kick in when an excessive number of child process crashes are observed. Also, we may use the stack trace from the crash dump to guide the selection of fuzzing entry points. Static analysis and performance profiling information (e.g., hotspots) of the target program could also assist in selecting suitable fuzzing entry points.

Under the multi-container mode, the fuzzing phase can be initiated from a history checkpoint of the target program. The fuzzing phase can thus be delayed to off-peak hours to balance resource usage further. This may also be useful to forensics applications, where In-Vivo Fuzzing can help understand the dynamics of suspected attack activities in retrospect.

X. Conclusion

We proposed In-Vivo Fuzzing to realize online fuzz testing on live programs. This was motivated by the observation that real-world programs and their workloads are becoming more complex. Furthermore, both the environment setup and the operation of offline fuzz testing depend on domain knowledge substantially. In-Vivo Fuzzing takes a trade-off by accepting the presence of outstanding vulnerabilities in production software and embracing the opportunities of online fuzz testing on live programs. We built a prototype based on the AFL fuzzing engine. The evaluation with representative network service programs and CVEs demonstrates the efficacy of In-Vivo Fuzzing. With proper selection of fuzzing entry points and test seeds, a crash can be triggered within minutes on the same system platform running the network service program. With In-Vivo Fuzzing, the test data are injected directly via the I/O library call hooks, alleviating the need to prepare test harness code. Overall, In-Vivo Fuzzing makes fuzz testing more accessible. System administrators or end-users can deploy In-Vivo Fuzzing to look for vulnerabilities continuously in deployed systems. We believe this will be a crucial next step in taming the security threats faced by today's network services.

Acknowledgment

This study is supported by the Ministry of Science and Technology of the Republic of China under grant number 110-2628-E-A49-004 and 111-2218-E-A49-013-MBK.

Reference

[1] G. Klees, A. Ruef, B. Cooper, S. Wei, and M. Hicks, "Evaluating fuzz testing," in Proceedings of the 25th ACM SIGSAC Conference on Computer and Communications Security, 2018, pp. 2123-2138.

[2] Y. Zheng, A. Davanian, H. Yin, C. Song, H. Zhu, and L. Sun, "FIRM-AFL: high-throughput greybox fuzzing

of iot firmware via augmented process emulation," in Proceedings of the 28th USENIX Security Symposium, 2019, pp. 1099-1114.
[3] S. Schumilo, C. Aschermann, R. Gawlik, S. Schinzel, and T. Holz, "kafl: Hardware-assisted feedback fuzzing for {OS} kernels," in Proceedings of the 26th USENIX Security Symposium, 2017, pp. 167-182.
[4] V.-T. Pham, M. Böhme, and A. Roychoudhury, "AFLNET: A Greybox Fuzzer for Network Protocols," in The 13th International Conference on Software Testing, Verification and Validation (Testing Tools Track), 2020: IEEE.
[5] J. Chen et al., "IoTFuzzer: Discovering Memory Corruptions in IoT Through App-based Fuzzing," in The Network and Distributed System Security Symposium, 2018: Internet Society.
[6] D. R. Jeong, K. Kim, B. Shivakumar, B. Lee, and I. Shin, "Razzer: Finding kernel race bugs through fuzzing," in Proceedings of the 40th IEEE Symposium on Security and Privacy, 2019: IEEE, pp. 754-768.
[7] S. Rawat, V. Jain, A. Kumar, L. Cojocar, C. Giuffrida, and H. Bos, "VUzzer: Application-aware Evolutionary Fuzzing," in Proceedings of the Network and Distributed System Security Symposium 2017, vol. 17: Internet Society, pp. 1-14.
[8] S. Gan et al., "Collafl: Path sensitive fuzzing," in Proceedings of the 39th IEEE Symposium on Security and Privacy, 2018: IEEE, pp. 679-696.
[9] P. Zong, T. Lv, D. Wang, Z. Deng, R. Liang, and K. Chen, "Fuzzguard: Filtering out unreachable inputs in directed grey-box fuzzing through deep learning," in Proceedings of the 29th USENIX Security Symposium, 2020, pp. 2255-2269.
[10] M. Böhme, V.-T. Pham, M.-D. Nguyen, and A. Roychoudhury, "Directed greybox fuzzing," in Proceedings of the ACM SIGSAC Conference on Computer and Communications Security, 2017, pp. 2329-2344.
[11] D. She, K. Pei, D. Epstein, J. Yang, B. Ray, and S. Jana, "NEUZZ: Efficient fuzzing with neural program smoothing," in Proceedings of the 40th IEEE Symposium on Security and Privacy, 2019: IEEE, pp. 803-817.
[12] H. Peng, Y. Shoshitaishvili, and M. Payer, "T-Fuzz: fuzzing by program transformation," in Proceedings of the 39th IEEE Symposium on Security and Privacy, 2018: IEEE, pp. 697-710.
[13] I. Yun, S. Lee, M. Xu, Y. Jang, and T. Kim, "{QSYM}: A practical concolic execution engine tailored for hybrid fuzzing," in Proceedings of the 27th USENIX Security Symposium, 2018, pp. 745-761.
[14] K. Ispoglou, D. Austin, V. Mohan, and M. Payer, "FuzzGen: Automatic Fuzzer Generation," in Proceedings of the 29th USENIX Security Symposium, 2020.
[15] K. Serebryany, "libFuzzer–a library for coverage-guided fuzz testing," LLVM project, 2015.
[16] N. Stephens et al., "Driller: Augmenting Fuzzing Through Selective Symbolic Execution," in Proceedings of the Network and Distributed System Security Symposium, 2016: Internet Society.
[17] J. Kutscher. "M-Trends 2021: A View From the Front Lines." FireEye Mandiant. https://www.mandiant.com/resources/m-trends-2021-a-view-from-the-front-lines (accessed 12-6, 2021).
[18] S. Radack, "The system development life cycle (sdlc)," National Institute of Standards and Technology, 2009. [Online]. Available: https://csrc.nist.gov/csrc/media/publications/shared/documents/itl-bulletin/itlbul2009-04.pdf
[19] W. Drewry. "Dynamic seccomp policies (using BPF filters)." https://lwn.net/Articles/475019/ (accessed 11-12, 2021).
[20] The Containers Organization. "The Pod Manager Tool " https://podman.io/ (accessed 4-11, 2022).
[21] OpenVZ Team at Virtuozzo. "Checkpoint/Restore In Userspace." https://criu.org (accessed 12-10, 2021).
[22] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in Proceedings of the International Symposium on Code Generation and Optimization, 2004: IEEE, pp. 75-86.
[23] R. M. Stallman, Using and porting the GNU compiler collection. Free Software Foundation, 1999.
[24] W. Glozer. "wrk - a HTTP benchmarking tool." https://github.com/wg/wrk (accessed 12-11, 2021).
[25] A. Kopytov. "sysbench." https://github.com/akopytov/sysbench (accessed 2/11, 2021).
[26] J. Corina et al., "Difuze: Interface aware fuzzing for kernel drivers," in Proceedings of the ACM SIGSAC Conference on Computer and Communications Security, 2017, pp. 2123-2138.
[27] Z. Sudhackar. "preeny." https://github.com/zardus/preeny (accessed 11-1, 2021).
[28] Y. Chen, T. Lan, and G. Venkataramani, "Exploring effective fuzzing strategies to analyze communication protocols," in Proceedings of the 3rd ACM Workshop on Forming an Ecosystem Around Software Transformation, 2019, pp. 17-23.
[29] D. Oleksiuk. "IOCTL Fuzzer." eSage lab. https://github.com/Cr4sh/ioctlfuzzer (accessed 3-27, 2022).
[30] D. Laws. "kuzz." https://github.com/Haifisch/kuzz (accessed 3-27, 2022).
[31] M. Li. "PassiveFuzzFrameworkOSX." https://github.com/SilverMoonSecurity/PassiveFuzzFrameworkOSX (accessed 3-27, 2022).
[32] H. Liang, Y. Chen, Z. Xie, and Z. Liang, "X-afl: A kernel fuzzer combining passive and active fuzzing," in Proceedings of the 13th European workshop on Systems Security, 2020, pp. 13-18.